

Walkthroughs and Exercises for *Statistical Modeling and Inference with Python*

Dr. Chester Ismay

```
import pandas as pd

# Display all columns
pd.set_option('display.max_columns', None)

# Ensure printing of all output for each cell
from IPython.core.interactiveshell import InteractiveShell
InteractiveShell.ast_node_interactivity = "all"
```

Week 1

Walkthrough 1.1: Getting Started

Setting Up the Python Environment

If you haven't already installed Python, Jupyter, and the necessary packages, there are instructions on the course repo in the README to do so here.

If you aren't able to do this on your machine, you may want to check out Google Colab. It's a free service that allows you to run Jupyter notebooks in the cloud.

```
# Importing libraries/modules and aliasing them as needed
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import statsmodels.api as sm
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score
```

Load a dataset

```
# Load in the dataset
spotify_sample = pd.read_csv("spotify_sample.csv")
```

Prepare data

```
# Select a single feature and target variable

# Split the data into training and testing sets
```

Train and use the model

```
# Create and train the model

# Make predictions

# Evaluate the model

# Plotting the results
```

Check if assumptions of linear regression met with visual tools

```
# Check for Homoscedasticity

# Check for Normality of Residuals
```

Exercise 1.1: Getting Started

Setting Up the Python Environment

If you ran the `# Importing libraries and aliasing them` code above, you should be good to proceed here. If not, scroll up and run it.

Load a dataset

```
# Load in the dataset
imdb_movie_sample = pd.read_csv("imdb_movie_sample.csv")
```

Prepare data

```
# Select a single feature (votes) and target variable (rating)

# Split the data into training and testing (0.2 size) sets
# with seed of 2024
```

Train and use the model

```
# Create and train the model

# Make predictions

# Evaluate the model
```

```
# Plotting the results
```

Check if assumptions of linear regression met with visual tools

```
# Check for Homoscedasticity
```

```
# Check for Normality of Residuals
```

Walkthrough 1.2: Correlation

Correlation matrix

```
# Select only numeric columns
```

```
# Calculate Pearson correlation matrix
```

```
# Display the correlation matrix
```

Visualizing correlation matrix

```
# Visualize the correlation matrix using a heatmap
```

Visualizing relationships

```
# Plot pairplot to visualize relationships
```

```
# between 'danceability', 'energy', 'loudness', 'valence', 'popularity'
```

Exercise 1.2: Correlation

Correlation matrix

```
# Select only numeric columns
```

```
# Calculate Pearson correlation matrix
```

```
# Display the correlation matrix
```

Visualizing correlation matrix

```
# Visualize the correlation matrix using a heatmap
```

Visualizing relationships

```
# Plot pairplot to visualize relationships of  
# runtime_in_minutes, rating, votes, and gross_in_dollars
```

Walkthrough 1.3: Multiple Regression

```
# Prepare the data for multiple regression  
  
# Add a constant to the predictor variable set  
  
# Fit the multiple regression model  
  
# Print the model summary
```

Exercise 1.3: Multiple Regression

```
# Prepare the data for multiple regression choosing runtime_in_minutes, votes,  
# and gross_in_dollars as predictors and rating as the target variable  
  
# Add a constant to the predictor variable set  
  
# Fit the multiple regression model  
  
# Print the model summary
```

Walkthrough 1.4: Logistic Regression

```
# Define the predictors and response  
  
# Add a constant to the model (intercept)  
  
# Fit the logistic regression model  
  
# Print the summary  
  
# Predict probabilities  
  
# Plot the predicted probabilities
```

Exercise 1.4: Logistic Regression

```
# Define the predictors (runtime_in_minutes & votes) and response (rating > 7)

# Add a constant to the model (intercept)

# Fit the logistic regression model

# Print the summary

# Predict probabilities

# Plot the predicted probabilities
```

Walkthrough 1.5: ANOVA

One-way ANOVA

```
from statsmodels.formula.api import ols

# Choose some genres to compare

# Subset the data to focus only on these genres

# Perform one-way ANOVA analyzing energy across different genres
```

Boxplot across groups

```
# Boxplot for visualization

# Adjust layout to prevent labels from being cut off

# Show the plot
```

Two-way ANOVA

```
# Perform two-way ANOVA analyzing energy across genres and explicit content
```

Boxplot across groups with color added

```
# Boxplot for visualization
```

Exercise 1.5: ANOVA

One-way ANOVA

```
# Choose some genres to compare ('Action', 'Crime', 'Horror', 'Romance')  
  
# Subset the data to focus only on these genres  
  
# Perform one-way ANOVA analyzing rating across different genres
```

Boxplot across groups

```
# Boxplot for visualization
```

Two-way ANOVA

```
# Perform two-way ANOVA analyzing rating across genres and decades
```

Boxplot across groups with color added

```
# Some customization here to get the decades to appear in the appropriate order  
  
# Define the ordered categories  
decades = ['1940s', '1950s', '1960s', '1970s', '1980s', '1990s', '2000s', '2010s', '2020s']  
  
# Convert 'decade' column to a categorical type with ordered categories  
imdb_movie_sample_subset['decade'] = pd.Categorical(  
    imdb_movie_sample_subset['decade'],  
    categories=decades,  
    ordered=True)  
  
# Boxplot for visualization
```

Walkthrough 2.1: Kruskal-Wallis and Mann-Whitney U Tests

Kruskal-Wallis

```
import scipy.stats as stats  
  
# Perform Kruskal-Wallis Test on 'danceability' grouped by 'track_genre'
```

Mann-Whitney U

```
# Perform Mann-Whitney U Test on 'energy' for pop versus rock
```

Exercise 2.1: Kruskal-Wallis and Mann-Whitney U Tests

Kruskal-Wallis

```
# Perform Kruskal-Wallis Test on 'rating' grouped by 'decade'
```

Mann-Whitney U

```
# Perform Mann-Whitney U Test on 'gross_in_dollars' for 1990s versus 2000s
```

Walkthrough 2.2: Correlation and Regression Non-parametrics

Spearman's Rank Correlation

```
from scipy.stats import spearmanr, kendalltau
from sklearn.linear_model import TheilSenRegressor

# Filter numeric columns

# Calculate Spearman's rank correlation matrix

# Heatmap for Spearman's rank correlation matrix
```

Kendall's Tau Correlation

```
# Calculate Kendall's tau correlation matrix

# Heatmap for Kendall's tau correlation matrix
```

Theil-Sen Robust Regression

```
# Theil-Sen robust regression for 'danceability' and
# 'energy' predicting 'popularity'
```

Exercise 2.2: Correlation and Regression Non-parametrics

Spearman's Rank Correlation

```
# Filter numeric columns

# Calculate Spearman's rank correlation matrix
```

```
# Heatmap for Spearman's rank correlation matrix
```

Kendall's Tau Correlation

```
# Calculate Kendall's tau correlation matrix
```

```
# Heatmap for Kendall's tau correlation matrix
```

Theil-Sen Robust Regression

```
# Theil-Sen robust regression for 'votes' and 'runtime_in_minutes'  
# predicting 'rating'
```

Walkthrough 2.3: Bootstrapping

```
# Function to perform bootstrapping
```

```
# Perform bootstrapping on the 'energy' column
```

```
# Plot the distribution of bootstrap means
```

Exercise 2.3: Bootstrapping

```
# Perform bootstrapping on the 'runtime_in_minutes' column  
# using the bootstrap function defined above
```

```
# Plot the distribution of bootstrap means
```

Walkthrough 2.4: Confidence Intervals

```
# Function to calculate bootstrap confidence intervals
```

```
# Calculate confidence intervals for the 'energy' column
```

```
# Plot the distribution of bootstrap means and confidence interval
```

Exercise 2.4: Confidence Intervals

```
# Calculate confidence intervals for the 'rating' column using the  
# bootstrap_confidence_interval function defined above
```



```
# Plot the distribution of bootstrap means and confidence interval
```

Walkthrough 2.5: Real-World Scenarios for Bootstrapping

Music Industry Analytics

Objective: Analyzing song popularity and estimating population mean popularity.

Scenario: You are a data scientist for a movie production company. Your role is to analyze movie ratings and predict the average rating of movies produced. By using bootstrapping techniques, you aim to estimate the mean movie rating and its variability to understand the unknown population mean.

Bootstrapping Applications:

- *Estimating Average Popularity:* Use bootstrapping to calculate the mean and confidence intervals of the popularity scores.
- *Assessing Variability:* Determine the variability in the means of song popularity.

```
# Calculate bootstrap statistics for the 'popularity' column
```

```
# Function to calculate bootstrap statistics
```

```
# Plot the distribution of bootstrap means
```

Exercise 2.5: Real-World Scenarios for Bootstrapping

Film Industry Insights

Objective: Evaluating movie gross revenue and estimating population mean gross revenue.

Scenario: You are a data scientist for a movie production company. Your role is to analyze movie gross revenue and predict the average gross revenue of movies produced. By using bootstrapping techniques, you aim to estimate the mean gross revenue and its variability to understand the unknown population mean.

Bootstrapping Applications:

- *Estimating Average Gross Revenue:* Use bootstrapping to calculate the mean and confidence intervals of movie gross revenue.
- *Assessing Variability:* Determine the variability in the means of movie gross revenue.

```
# Calculate bootstrap statistics for the 'gross_in_dollars' column
```

```
# using the function called bootstrap_statistics defined above
```

```
# Plot the distribution of bootstrap means
```